rCore-Tutorial-v3 内核实现学习报告

一、概述

1.1 rCore 简介

rCore-Tutorial-v3 是一个基于 RISC-V 架构的教学操作系统，使用 Rust 语言编写。它从零开始实现了一个功能完整的类 Unix 内核，涵盖了操作系统核心概念：进程管理、内存管理、文件系统、进程间通信等。

1.2 开发环境

组件	版本/说明
开发语言	Rust (nightly)
目标架构	RISC-V 64位 (rv64gc)
模拟器	QEMU 7.0.0
Bootloader	RustSBI 0.3.1
工具链	riscv64-unknown-elf-gcc

1.3 章节结构概览

章节	主题	核心功能
Ch1	独立内核栈	最小内核启动、SBI 调用
Ch2	批处理系统	Trap 机制、系统调用
Ch3	多道程序	任务切换、协作式调度
Ch4	虚拟内存	地址空间、页表管理
Ch5	文件系统	easy-fs、inode 抽象
Ch6	进程管理	进程/线程、同步原语

二、内核启动流程（Chapter 1）

2.1 启动过程概述

```
┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│  QEMU 启动   │ ──▶ │  RustSBI     │ ──▶ │  内核入口    │
│  0x1000      │    │  0x80000000  │    │  0x80200000  │
└──────────────┘    └──────────────┘    └──────────────┘
```

2.2 核心组件

2.2.1 链接脚本 (linker.ld)

```
OUTPUT_ARCH(riscv)
ENTRY(_start)
BASE_ADDRESS = 0x80200000;

SECTIONS {
    . = BASE_ADDRESS;
    .text : { *(.text.entry) *(.text .text.*) }
    .rodata : { *(.rodata .rodata.*) }
    .data : { *(.data .data.*) }
    .bss : { *(.bss .bss.*) }
}
```

关键点：

- 内核加载地址：`0x80200000`
- 定义各段内存布局：`.text` → `.rodata` → `.data` → `.bss`

2.2.2 汇编入口 (entry.asm)

```
.section .text.entry
.globl _start
_start:
    la sp, boot_stack_top
    call rust_main

.section .bss.stack
.globl boot_stack_lower_bound
boot_stack_lower_bound:
    .space 4096 * 16
.globl boot_stack_top
boot_stack_top:
```

功能：

- 设置内核栈指针 (sp)
- 跳转到 Rust 主函数 `rust_main`

2.2.3 SBI 调用封装 (sbi.rs)

```
pub fn sbi_call(eid: usize, fid: usize, arg0: usize, arg1: usize, arg2:
usize) -> usize {
    let mut ret;
    unsafe {
        asm!(
            "ecall",
```

```
            inlateout("x10") arg0 => ret,
            in("x11") arg1,
            in("x12") arg2,
            in("x16") fid,
            in("x17") eid,
        );
    }
    ret
}
```

说明：通过 `ecall` 指令调用 RustSBI 提供的服务（如字符输出、关机等）。

三、Trap 机制与系统调用（Chapter 2）

3.1 Trap 处理流程

```
┌──────────────┐
│  用户态程序  │ ── ecall/ecall ──▶ │  __alltraps  │
└──────────────┘                    └──────────────┘
                                          │
                                          ▼
                                    ┌──────────────┐
                                    │ trap_handler │
                                    └──────────────┘
                                          │
                        ┌─────────────────┼─────────────────┐
                        ▼                 ▼                 ▼
                  ┌──────────┐       ┌──────────┐       ┌──────────┐
                  │ 系统调用 │       │ 页错误   │       │ 时钟中断 │
                  └──────────┘       └──────────┘       └──────────┘
```

3.2 Trap 上下文结构

```rust
pub struct TrapContext {
    pub x: [usize; 32],   // 通用寄存器
    pub sstatus: Sstatus, // 状态寄存器
    pub sepc: usize,      // 异常程序计数器
    pub kernel_satp: usize, // 内核页表
    pub kernel_sp: usize,   // 内核栈指针
    pub trap_handler: usize, // trap 处理函数地址
}
```

3.3 Trap 处理函数

```
#[unsafe(no_mangle)]
pub fn trap_handler(cx: &mut TrapContext) -> &mut TrapContext {
    let scause = scause::read();
    match scause.cause() {
        Trap::Exception(Exception::UserEnvCall) => {
            cx.sepc += 4;  // 跳过 ecall 指令
            cx.x[10] = syscall(cx.x[17], [cx.x[10], cx.x[11], cx.x[12]])
as usize;
        }
        Trap::Exception(Exception::StoreFault) |
Trap::Exception(Exception::StorePageFault) => {
            println!("[kernel] PageFault in application, kernel killed
it.");
            run_next_app();
        }
        // ... 其他异常处理
    }
    cx
}
```

3.4 系统调用分发

```
pub fn syscall(syscall_id: usize, args: [usize; 3]) -> isize {
    match syscall_id {
        SYSCALL_WRITE => sys_write(args[0], args[1] as *const u8,
args[2]),
        SYSCALL_EXIT => sys_exit(args[0] as i32),
        SYSCALL_YIELD => sys_yield(),
        // ... 其他系统调用
        _ => panic!("Unsupported syscall: {}", syscall_id),
    }
}
```

四、任务管理与调度（Chapter 3）

4.1 任务控制块 (TCB)

```
pub struct TaskControlBlock {
    pub process: Weak<ProcessControlBlock>,  // 所属进程
    pub kstack: KernelStack,                  // 内核栈
    pub inner: UPIntrFreeCell<TaskControlBlockInner>,
}

pub struct TaskControlBlockInner {
    pub res: Option<TaskUserRes>,    // 用户资源
    pub trap_cx_ppn: PhysPageNum,    // Trap 上下文物理页
    pub task_cx: TaskContext,         // 任务上下文
```

```
    pub task_status: TaskStatus,      // 任务状态
    pub exit_code: Option<i32>,       // 退出码
}
```

4.2 任务状态转换

```
┌─────────┐
│ Ready   │◄──────────────────┐
└─────────┘                   │
    │ schedule                │
    ▼                         │
┌─────────┐  block/wait       │
│Running  │ ─────────────────►│
└─────────┘                   │
    │ exit                    │
    ▼                         │
┌─────────┐                 ┌─────────┐
│Exited   │                 │ Blocked │
└─────────┘                 └─────────┘
```

4.3 任务切换机制

```
pub fn __switch(current_task_cx: &mut TaskContext, next_task_cx:
&TaskContext) {
    // 保存当前任务上下文
    // 恢复下一个任务上下文
}
```

关键汇编实现 (switch.S)：

```
__switch:
    sd ra, 0(a0)
    sd sp, 8(a0)
    sd s0, 16(a0)
    # ... 保存 callee-saved 寄存器

    ld ra, 0(a1)
    ld sp, 8(a1)
    ld s0, 16(a1)
    # ... 恢复 callee-saved 寄存器
    ret
```

五、虚拟内存管理（Chapter 4）

5.1 地址空间抽象

```
pub struct MemorySet {
    page_table: PageTable,      // 页表
    areas: Vec<MapArea>,        // 虚拟内存区域
}
```

5.2 内存映射区域

```
pub struct MapArea {
    vpn_range: VPNRange,                    // 虚拟页号范围
    data_frames: BTreeMap<VirtPageNum, FrameTracker>, // 物理帧
    map_type: MapType,                      // 映射类型
    map_perm: MapPermission,                // 权限
}
```

5.3 映射类型

```
pub enum MapType {
    Identical,            // 恒等映射：虚拟地址 = 物理地址
    Framed,               // 按帧分配：动态分配物理页
    Linear(isize),        // 线性映射：带偏移
}
```

5.4 权限控制

```
bitflags! {
    pub struct MapPermission: u8 {
        const R = 1 << 1;  // 可读
        const W = 1 << 2;  // 可写
        const X = 1 << 3;  // 可执行
        const U = 1 << 4;  // 用户态可访问
    }
}
```

5.5 内核地址空间初始化

```
pub fn new_kernel() -> Self {
    let mut memory_set = Self::new_bare();

    // 映射跳板页
    memory_set.map_trampoline();
```

```
    // 映射内核各段（恒等映射）
    memory_set.push(MapArea::new(
        (stext as usize).into(),
        (etext as usize).into(),
        MapType::Identical,
        MapPermission::R | MapPermission::X,  // 代码段：可读可执行
    ), None);

    // ... 其他段映射

    memory_set
}
```

5.6 用户地址空间创建（从 ELF）

```
pub fn from_elf(elf_data: &[u8]) -> (Self, usize, usize) {
    let mut memory_set = Self::new_bare();
    memory_set.map_trampoline();

    let elf = xmas_elf::ElfFile::new(elf_data).unwrap();

    // 解析 ELF 程序头，映射各段
    for i in 0..ph_count {
        let ph = elf.program_header(i).unwrap();
        if ph.get_type().unwrap() == Type::Load {
            // 根据 ELF 标志设置权限
            let map_perm = MapPermission::U | ...;
            memory_set.push(map_area, Some(data));
        }
    }

    (memory_set, user_stack_base, entry_point)
}
```

六、文件系统（Chapter 5）

6.1 文件系统架构

```
┌──────────────────────────────────────────────────┐
│                    应用程序                       │
└──────────────────────────────────────────────────┘
                          │
                          │ syscall (read/write)
                          ▼
┌──────────────────────────────────────────────────┐
│            OSInode (内核文件抽象)                 │
└──────────────────────────────────────────────────┘
                          │
                          │
                          ▼
```

```
┌──────────────────────────────────────────────────────┐
│            Inode (easy-fs 核心结构)                   │
└──────────────────────────────────────────────────────┘
                          │
                          │
            ┌─────────────┼─────────────┐
            ▼             ▼             ▼
      ┌──────────┐   ┌──────────┐   ┌──────────┐
      │   Data   │   │   inode  │   │  Bitmap  │
      │  Blocks  │   │   Table  │   │          │
      └──────────┘   └──────────┘   └──────────┘
                          │
                          ▼
                 ┌──────────────────┐
                 │   Block Device   │
                 │   (virtio-blk)   │
                 └──────────────────┘
```

6.2 OSInode 结构

```rust
pub struct OSInode {
    readable: bool,
    writable: bool,
    inner: UPIntrFreeCell<OSInodeInner>,
}

pub struct OSInodeInner {
    offset: usize,            // 当前读写偏移
    inode: Arc<Inode>,        // 底层 easy-fs inode
}
```

6.3 文件打开实现

```rust
pub fn open_file(name: &str, flags: OpenFlags) -> Option<Arc<OSInode>> {
    let (readable, writable) = flags.read_write();

    if flags.contains(OpenFlags::CREATE) {
        if let Some(inode) = ROOT_INODE.find(name) {
            inode.clear();  // 清空文件
            Some(Arc::new(OSInode::new(readable, writable, inode)))
        } else {
            ROOT_INODE.create(name)  // 创建新文件
                .map(|inode| Arc::new(OSInode::new(readable, writable,
inode)))
        }
    } else {
        ROOT_INODE.find(name).map(|inode| {
            if flags.contains(OpenFlags::TRUNC) {
                inode.clear();
            }
```

```
            Arc::new(OSInode::new(readable, writable, inode))
        })
    }
}
```

6.4 文件读写操作

```
impl File for OSInode {
    fn read(&self, mut buf: UserBuffer) -> usize {
        let mut inner = self.inner.exclusive_access();
        let mut total_read_size = 0usize;

        for slice in buf.buffers.iter_mut() {
            let read_size = inner.inode.read_at(inner.offset, *slice);
            if read_size == 0 { break; }
            inner.offset += read_size;
            total_read_size += read_size;
        }
        total_read_size
    }

    fn write(&self, buf: UserBuffer) -> usize {
        let mut inner = self.inner.exclusive_access();
        let mut total_write_size = 0usize;

        for slice in buf.buffers.iter() {
            let write_size = inner.inode.write_at(inner.offset, *slice);
            inner.offset += write_size;
            total_write_size += write_size;
        }
        total_write_size
    }
}
```

七、进程管理（Chapter 6）

7.1 进程控制块

```
pub struct ProcessControlBlock {
    pub pid: usize,                                // 进程 ID
    pub inner: UPIntrFreeCell<ProcessControlBlockInner>,
}

pub struct ProcessControlBlockInner {
    pub memory_set: MemorySet,            // 地址空间
    pub parent: Option<Weak<ProcessControlBlock>>, // 父进程
    pub children: Vec<Arc<ProcessControlBlock>>,   // 子进程列表
    pub tasks: Vec<Option<Arc<TaskControlBlock>>>, // 线程列表
```

```
    pub exit_code: i32,                          // 退出码
}
```

7.2 进程创建流程

```
┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│ fork 系统调用 │ ──▶ │ 复制进程控制块 │ ──▶ │ 复制地址空间  │
└──────────────┘    └──────────────┘    └──────────────┘
                                                │
                                                ▼
                    ┌──────────────┐    ┌──────────────┐
                    │ 返回子进程PID │ ◀── │ 复制文件描述符 │
                    └──────────────┘    └──────────────┘
```

7.3 同步原语

7.3.1 互斥锁 (Mutex)

```rust
pub struct Mutex {
    inner: UPIntrFreeCell<MutexInner>,
}

pub struct MutexInner {
    locked: bool,
    wait_queue: VecDeque<Arc<TaskControlBlock>>,
}
```

7.3.2 信号量 (Semaphore)

```rust
pub struct Semaphore {
    pub inner: UPIntrFreeCell<SemaphoreInner>,
}

pub struct SemaphoreInner {
    pub count: isize,
    pub wait_queue: VecDeque<Arc<TaskControlBlock>>,
}
```

7.3.3 条件变量 (Condvar)

```rust
pub struct Condvar {
    pub inner: UPIntrFreeCell<CondvarInner>,
}
```

```rust
pub struct CondvarInner {
    pub wait_queue: VecDeque<Arc<TaskControlBlock>>,
}
```

7.4 进程间通信

7.4.1 管道 (Pipe)

```rust
pub struct Pipe {
    readable: bool,
    writable: bool,
    buffer: [u8; RING_BUFFER_SIZE],
    head: usize,
    tail: usize,
    status: RingBufferStatus,
}
```

八、关键代码分析

8.1 任务调度核心流程

```rust
pub fn run_tasks() {
    loop {
        let task = fetch_task();  // 从就绪队列获取任务
        if let Some(task) = task {
            // 获取当前任务
            let current_task = CURRENT_TASK.exclusive_access();

            // 切换到新任务
            unsafe {
                __switch(
                    &mut current_task.inner_exclusive_access().task_cx,
                    &task.inner_exclusive_access().task_cx,
                );
            }
        }
    }
}
```

8.2 系统调用处理

系统调用号	名称	功能
64	sys_write	写入文件/标准输出

系统调用号	名称	功能
93	sys_exit	进程退出
124	sys_yield	主动让出 CPU
214	sys_fork	创建子进程
215	sys_exec	执行新程序
216	sys_waitpid	等待子进程

九、项目结构

```
rCore-Tutorial-v3-ch6/
├── os/                          # 内核实现
│   └── src/
│       ├── main.rs              # 内核入口
│       ├── config.rs            # 配置常量
│       ├── console.rs           # 控制台输出
│       ├── sbi.rs               # SBI 调用封装
│       ├── timer.rs             # 时钟管理
│       ├── entry.asm            # 汇编入口
│       ├── linker-qemu.ld       # 链接脚本
│       ├── trap/                # Trap 处理
│       │   ├── mod.rs
│       │   ├── context.rs
│       │   └── trap.S
│       ├── task/                # 任务管理
│       │   ├── mod.rs
│       │   ├── task.rs
│       │   ├── process.rs
│       │   ├── manager.rs
│       │   ├── processor.rs
│       │   └── switch.S
│       ├── mm/                  # 内存管理
│       │   ├── mod.rs
│       │   ├── address.rs
│       │   ├── page_table.rs
│       │   ├── memory_set.rs
│       │   └── frame_allocator.rs
│       ├── fs/                  # 文件系统
│       │   ├── mod.rs
│       │   ├── inode.rs
│       │   ├── pipe.rs
│       │   └── stdio.rs
│       ├── sync/                # 同步原语
│       │   ├── mod.rs
│       │   ├── mutex.rs
│       │   ├── semaphore.rs
│       │   └── condvar.rs
│       ├── syscall/             # 系统调用
```

```
│       │   ├── mod.rs
│       │   ├── process.rs
│       │   ├── fs.rs
│       │   └── sync.rs
│       └── drivers/              # 设备驱动
│           ├── plic.rs
│           └── block/virtio_blk.rs
├── user/                         # 用户程序
├── easy-fs/                      # 简易文件系统
└── bootloader/                   # RustSBI 固件
```

十、学习心得

10.1 核心收获

1. **系统化理解操作系统**：通过从零实现内核，深入理解了进程、内存、文件系统等核心概念
2. **Rust 在系统编程中的优势**：
   - 所有权系统保证内存安全
   - 无运行时的特性适合裸机开发
   - 强类型系统减少运行时错误
3. **RISC-V 架构特点**：
   - 简洁的指令集设计
   - 清晰的特权级划分
   - 规范的 SBI 接口

10.2 关键技术点

技术点	难点	理解
Trap 上下文	上下文保存/恢复	理解用户态/内核态切换
虚拟内存	页表映射	理解地址空间隔离
任务切换	上下文切换	理解协程/线程调度
文件系统	inode 抽象	理解 Unix 文件模型

10.3 后续展望

- 深入学习多核调度（SMP）
- 研究更复杂的文件系统实现
- 探索网络协议栈实现
- 学习设备驱动开发

参考资料

- rCore-Tutorial-v3 官方文档
- RISC-V 特权架构规范
- Rust 嵌入式开发指南